

	FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL	CÓDIGO: TI-SIGE-ENT-S2-006
	CARRERA DE INGENIERÍA DE SOFTWARE	VERSIÓN: 1.0
	PROCESO: INGENIERÍA DE SOFTWARE / DEVOPS E INFRAESTRUCTURA	ELABORACIÓN: 04/09/2026
	ARQUITECTURA DE CONTENEDORES, ORQUESTACIÓN CON DOCKER COMPOSE Y DESPLIEGUE LOCAL	ÚLTIMA REVISIÓN: PENDIENTE DE REVISIÓN

1. TÍTULO

Denominación del Entregable: Configuración de Infraestructura como Código (IaC), Contenedores y Docker Compose (EDT 2.6)

Ficha Técnica de Identificación:

- **Código Documental:** TI-SIGE-ENT-S2-006 — **Versión:** 1.0
- **Institución:** Universidad Técnica de Ambato (UTA) – FISEI
- **Carrera:** Ingeniería de Software (Séptimo Semestre .A'')
- **Módulo Académico:** Gestión de Proyectos de Software
- **Docente Tutor:** Ing. Paulo Torres
- **Fecha de Elaboración:** 04/09/2026 — **Estado:** **En Elaboración / Pendiente de Revisión**
- **Período de Ejecución:** Semana 2 (02/09/2026 al 04/09/2026)
- **Responsables:** Steeven Toala (Líder / Arquitecto) y Nixon Hurtado (Backend / DBA)

2. INTRODUCCIÓN Y ESTRATEGIA DE CONTENEDORES

Para garantizar reproducibilidad en los entornos de desarrollo, pruebas y producción sin incurrir en costos de licencias comerciales ni saturar los recursos de hardware, el sistema **SIGE** implementa una arquitectura eficiente y desacoplada mediante **Docker Compose**:

- **Servicios Locales Contenerizados:** Base de datos relacional (**PostgreSQL 16**), almacenamiento de objetos S3 (**MinIO**), corrector ortográfico/gramatical local de alta velocidad (**LanguageTool**), backend (**ASP.NET Core .NET 9**) y cliente web (**React 19 + Nginx**).
- **Asistencia Semántica de Inteligencia Artificial:** En lugar de ejecutar un contenedor local pesado de Ollama (que requeriría descargar modelos de varios gigabytes y consumir entre 8 y 16 GB de RAM con soporte GPU inviable en infraestructura económica), la inferencia semántica para redacción institucional se delega a la **API de Google Gemini (Gemini 1.5 / 2.0 Flash)** a través de peticiones seguras HTTPS (REST API), optimizando la infraestructura local a cero sobre costo de hardware.

3. ESPECIFICACIÓN DEL ARCHIVO DOCKER-COMPOSE.YML

```
version: '3.8'

services:
```

	FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL	CÓDIGO: TI-SIGE-ENT-S2-006
	CARRERA DE INGENIERÍA DE SOFTWARE	VERSIÓN: 1.0
	PROCESO: INGENIERÍA DE SOFTWARE / DEVOPS E INFRAESTRUCTURA	ELABORACIÓN: 04/09/2026
	ARQUITECTURA DE CONTENEDORES, ORQUESTACIÓN CON DOCKER COMPOSE Y DESPLIEGUE LOCAL	ÚLTIMA REVISIÓN: PENDIENTE DE REVISIÓN

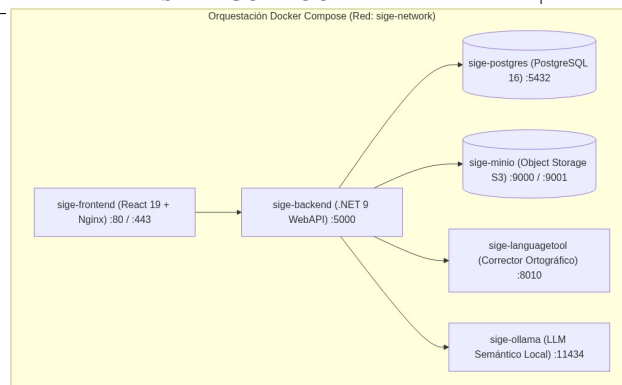


Figura 1: Arquitectura de Contenedores y Mapeo de Puertos en Docker Compose.

```

# 1. Base de Datos Relacional PostgreSQL 16
sige-postgres:
  image: postgres:16-alpine
  container_name: sige-postgres
  restart: unless-stopped
  environment:
    POSTGRES_DB: sige_db
    POSTGRES_USER: sige_admin
    POSTGRES_PASSWORD: SigSecurePassword2026!
  ports:
    - "5432:5432"
  volumes:
    - postgres_data:/var/lib/postgresql/data
  networks:
    - sige-network
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U sige_admin -d sige_db"]
    interval: 10s
    timeout: 5s
    retries: 5

# 2. Servidor de Objetos MinIO (Almacenamiento Seguro S3)
sige-minio:
  image: minio/minio:RELEASE.2024-05-10T01-41-38Z
  container_name: sige-minio
  restart: unless-stopped
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: sige_minio_admin
    MINIO_ROOT_PASSWORD: SigMinioSecretKey2026!
  ports:
    - "9000:9000" # API S3 Endpoint
    - "9001:9001" # Consola Web de Administracion
  volumes:
    - minio_data:/data
  networks:
    - sige-network

# 3. Inicializador de Bucket MinIO (Crea el bucket de evidencias automaticamente)
sige-minio-init:

```

	FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL	CÓDIGO: TI-SIGE-ENT-S2-006
	CARRERA DE INGENIERÍA DE SOFTWARE	VERSIÓN: 1.0
	PROCESO: INGENIERÍA DE SOFTWARE / DEVOPS E INFRAESTRUCTURA	ELABORACIÓN: 04/09/2026
	ARQUITECTURA DE CONTENEDORES, ORQUESTACIÓN CON DOCKER COMPOSE Y DESPLIEGUE LOCAL	ÚLTIMA REVISIÓN: PENDIENTE DE REVISIÓN

```

image: minio/mc:latest
container_name: sige-minio-init
depends_on:
  - sige-minio
entrypoint: >
  /bin/sh -c "
    until (/usr/bin/mc config host add sigeMinio http://sige-minio:9000
      sige_minio_admin SigeminioSecretKey2026!) do echo 'Esperando a MinIO...' &&
      sleep 2; done;
  /usr/bin/mc mb sigeMinio/sige-evidencias-bucket --ignore-existing;
  /usr/bin/mc anonymous set download sigeMinio/sige-evidencias-bucket;
  exit 0;
  "
networks:
  - sige-network

# 4. Servidor Local LanguageTool (Corrector Ortografico en Espanol)
sige-languagetool:
  image: erikvl87/languagetool:latest
  container_name: sige-languagetool
  restart: unless-stopped
  environment:
    langtool_languageModel: "/ngrams"
    Java_Xms: "512m"
    Java_Xmx: "1g"
  ports:
    - "8010:8010"
  networks:
    - sige-network

# 5. Backend ASP.NET Core (.NET 9 Web API)
sige-backend:
  build:
    context: ./src
    dockerfile: SIGE.WebAPI/Dockerfile
  container_name: sige-backend
  restart: unless-stopped
  depends_on:
    sige-postgres:
      condition: service_healthy
    sige-minio:
      condition: service_started
    sige-languagetool:
      condition: service_started
  environment:
    - ASPNETCORE_ENVIRONMENT=Development
    - ConnectionStrings__DefaultConnection=Host=sige-postgres;Port=5432;Database=sige_db;Username=sige_admin;Password=SigSecurePassword2026!;
    - MinIO__Endpoint=sige-minio:9000
    - MinIO__AccessKey=sige_minio_admin
    - MinIO__SecretKey=SigeminioSecretKey2026!
    - MinIO__BucketName=sige-evidencias-bucket
    - MinIO__UseSSL=false
    - LanguageTool__BaseUrl=http://sige-languagetool:8010
    - Gemini__ApiKey=${GEMINI_API_KEY}
    - Gemini__Model=gemini-1.5-flash
  ports:
    - "5000:8080"

```

	FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL	CÓDIGO: TI-SIGE-ENT-S2-006
	CARRERA DE INGENIERÍA DE SOFTWARE	VERSIÓN: 1.0
	PROCESO: INGENIERÍA DE SOFTWARE / DEVOPS E INFRAESTRUCTURA	ELABORACIÓN: 04/09/2026
	ARQUITECTURA DE CONTENEDORES, ORQUESTACIÓN CON DOCKER COMPOSE Y DESPLIEGUE LOCAL	ÚLTIMA REVISIÓN: PENDIENTE DE REVISIÓN

```

networks:
  - sige-network

# 6. Frontend React 19 + Nginx
sige-frontend:
  build:
    context: ./src/SIGE.ClientApp
    dockerfile: Dockerfile
  container_name: sige-frontend
  restart: unless-stopped
  depends_on:
    - sige-backend
  ports:
    - "80:80"
  networks:
    - sige-network

networks:
  sige-network:
    driver: bridge

volumes:
  postgres_data:
    driver: local
  minio_data:
    driver: local

```

4. COMANDOS DE DESPLIEGUE Y VERIFICACIÓN LOCAL

4.1. Inicio de Servicios

```

# 1. Clonar el repositorio y posicionarse en la raiz
cd d:/2Proyectos/GESTION_DE_PROYECTOS_DE_SOFTWARE/FISEI_Project

# 2. Levantar todos los contenedores en segundo plano
docker compose up -d

# 3. Verificar estado operativo de los servicios
docker compose ps

```

4.2. Endpoints de Verificación de Salud (Health Checks)

- **Frontend Web:** http://localhost:80
- **Swagger Web API:** http://localhost:5000/swagger
- **Consola MinIO:** http://localhost:9001 (User: sige_minio_admin/Pass: SigeminioSecretKey2026)
- **API LanguageTool:** http://localhost:8010/v2/check?text=Hola%20mundo&language=es
- **Asistente Gemini AI:** Conectividad HTTPS directa con https://generativelanguage.googleapis.com (verificable vía /api/ia/status en backend con API Key institucional).

	FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL	CÓDIGO: TI-SIGE-ENT-S2-006
	CARRERA DE INGENIERÍA DE SOFTWARE	VERSIÓN: 1.0
	PROCESO: INGENIERÍA DE SOFTWARE / DEVOPS E INFRAESTRUCTURA	ELABORACIÓN: 04/09/2026
	ARQUITECTURA DE CONTENEDORES, ORQUESTACIÓN CON DOCKER COMPOSE Y DESPLIEGUE LOCAL	ÚLTIMA REVISIÓN: PENDIENTE DE REVISIÓN

5. FIRMAS DE RESPONSABILIDAD

ACCIÓN	NOMBRE Y CARGO	FIRMA	FECHA
Elaborado por:	Steeven Toala <i>Líder de Proyecto / Arquitecto Software</i>		04/09/2026
Revisado por:	Steeven Toala <i>Líder de Proyecto / Arquitecto Software</i>	[PENDIENTE DE REVISIÓN]	Pendiente
Aprobado por:	Ing. Paulo Torres <i>Docente Tutor - Gestión Proyectos Software</i>	[PENDIENTE DE APROBACIÓN]	Pendiente

6. CONTROL DE CAMBIOS

Versión	Descripción del cambio	Responsable del cambio	Fecha de aprobación
1.0	Configuración de docker-compose.yml con los 6 servicios y comandos de verificación / Pendiente de revisión	Steeven Toala	Pendiente de Revisión

7. ANEXOS

Anexo 1: Script de Inicialización de Redes y Volúmenes de Docker

Comandos docker volume inspect y políticas de reinicio automático ante fallos de contenedor.